Status	Finished
Started	Wednesday, 2 September 2026, 7:28 PM
Completed	Wednesday, 2 September 2026, 7:30 PM
Duration	2 mins 13 secs
Marks	50.00/50.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct

Mark 1.00 out of 1.00

Which of the following are fundamental building blocks implemented directly in CPU hardware?

Select one:

- a. MUL and DIV
- b. ADD and SUB ✓
- c. Shift Left and Shift Right
- d. AND and OR

Your answer is correct.

The correct answer is: ADD and SUB

Question 2

Correct

Mark 1.00 out of 1.00

Early ARM CPUs (like ARM7TDMI) handled division by _____.

Select one:

- a. floating-point hardware
- b. pipeline flushing
- c. dedicated hardware unit
- d. compiler loops of simpler instructions ✓

Your answer is correct.

The correct answer is: compiler loops of simpler instructions

Question 3

Correct

Mark 1.00 out of 1.00

The shift-and-add method is used in _____.

Select one:

- a. Multiplication ✓
- b. Branch prediction
- c. Subtraction
- d. Division

Your answer is correct.

The correct answer is: Multiplication

Question 4

Correct

Mark 1.00 out of 1.00

Which classic algorithm is used in simple CPUs for division?

Select one:

- a. Shift-and-move
- b. Newton-Raphson
- c. Restoring division ✓
- d. Matrix inversion

Your answer is correct.

The correct answer is: Restoring division

Question 5

Correct

Mark 1.00 out of 1.00

Pipelining is often compared to _____.

Select one:

- a. a compiler optimization
- b. a recursive function
- c. an assembly line in a factory ✓
- d. a binary search tree

Your answer is correct.

The correct answer is: an assembly line in a factory

Question 6

Correct

Mark 1.00 out of 1.00

Which of the following is **not** a type of data hazard?

Select one:

- a. WAR
- b. RAM ✓
- c. RAW
- d. WAW

Your answer is correct.

The correct answer is: RAM

Question 7

Correct

Mark 1.00 out of 1.00

Which hazard occurs when a branch instruction disrupts the pipeline?

Select one:

- a. Control Hazard ✓
- b. Structural Hazard
- c. Data Hazard
- d. WAW Hazard

Your answer is correct.

The correct answer is: Control Hazard

Question 8

Correct

Mark 5.00 out of 5.00

```
add $s0, $t1, $t2   # Instruction 1
sub $t3, $s0, $t4   # Instruction 2
```

From the given instruction set, what data hazard can occur?

Select one:

- a. WAW
- b. RAR
- c. WAR
- d. RAW ✓

Your answer is correct.

The correct answer is: RAW

Question 9

Correct

Mark 8.00 out of 8.00

Consider this instruction set:

```
add $s0, $t1, $t2   # Instruction 1
sub $t3, $s0, $t4   # Instruction 2
```

It's possible to lead to the RAW ✓ **data hazard** when:

- **Instruction 1** produces a result in $s0 during its Write Back ✓ stage.
- **Instruction 2** needs the value of $s0 during its Instruction Decode ✓ stage.

But in the pipeline, the Instruction Decode ✓ of Instruction 2 happens before the Write Back ✓ of Instruction 1.

This means Instruction 2 tries to read ✓ $s0 before Instruction 1 has written ✓ the correct value into it.

Question 10

Correct

Mark 3.00 out of 3.00

Consider this instruction set:
lw $t0, 0($s0) # Instruction 1
add $t1, $t0, $s1 # Instruction 2
add $t0, $s2, $s3 # Instruction 3

Look at **$t0**:

In a simple in-order pipeline (like **MIPS**) → data hazards usually don't occur, because reads happen early in the Instruction Decode and writes happen late in the Write Back stage.

But in out-of-order pipelines (modern CPUs that reorder instructions for speed):
- Instruction 3 might be executed before Instruction 2's read completes.
- This creates a WAR hazard.

Question 11

Correct

Mark 1.00 out of 1.00

Which of the following best describes the difference between ***assembly language*** and ***machine language***?

Select one:

a. Assembly is portable across machines, while machine language is not.

b. Machine language is binary, while assembly uses a symbolic representation of instructions.

c. Machine language is higher-level than assembly.

d. Assembly is executed directly by the CPU, while machine language requires translation.

Your answer is correct.

The correct answer is: Machine language is binary, while assembly uses a symbolic representation of instructions.

Question 12

Correct

Mark 3.00 out of 3.00

What are the roles of the following MIPS registers?

- $t0–$t9: Temporary registers (caller-saved)
- $a0–$a3: Function/syscall arguments
- $v0: Syscall code or return values

Question 13

Correct

Mark 3.00 out of 3.00

What are the roles of the following MIPS registers?

- $t0–$t9: Temporary registers (caller-saved)
- $a0–$a3: Function/syscall arguments
- $v0: Syscall code or return values

Question 14

Correct

Mark 20.00 out of 20.00

```
.global _start
_start:

@ ARM Assembly:
@ Input: R1 = Multiplicand, R2 = Multiplier
@ Output: R0 = Result
  MOV R1, #12
  MOV R2, #20
  MOV R0, #0
  MOV R3, #32  @ Counter = 32 (bits)

mult_loop:
  ANDS R4, R2     @ Check the LSB of the Multiplier (R2)
  BEQ  no_add     @ If it's 0, skip adding
  ADD  R0, R0, R1 @ If it's 1, add Multiplicand to Result

no_add:
  LSL  R1, R1, #1 @ Shift Multiplicand left (double it)
  LSR  R2, R2, #1 @ Shift Multiplier right (next bit)
  SUBS R3, R3, #1 @ Decrement Counter
  BNE  mult_loop  @ If Counter !=0, loop again
@ R0 now contains R1 * R2
```

In shift-and-add algorithms:
Step 3: Look at the Least Significant Bit (LSB) of the Multiplier. (See steps in the slides).

Q1: What is the Multiplier in 32-bit data for the machine? (Please fill in each block of 4 bits)
Multiplier: 0000 ✓ 0000 ✓ 0000 ✓ 0000 ✓ 0000 ✓ 0000 ✓ 0001 ✓ 0100 ✓

Q2: What is the LSB of the Multiplier? 0 ✓

Q3: What to do next? LSL R1, R1 ✓

0101 | 0100 | 1000 | 1010 | 0001 | 0000

ANDS R4, R2 | ADD R0, R0, R1

1 | -1

Your answer is correct.

The correct answer is:

```
.global _start
_start:

@ ARM Assembly:
@ Input: R1 = Multiplicand, R2 = Multiplier
@ Output: R0 = Result
```

```
  MOV R1, #12
  MOV R2, #20
  MOV R0, #0
  MOV R3, #32  @ Counter = 32 (bits)

mult_loop:
  ANDS R4, R2     @ Check the LSB of the Multiplier (R2)
  BEQ  no_add     @ If it's 0, skip adding
  ADD  R0, R0, R1 @ If it's 1, add Multiplicand to Result

no_add:
  LSL  R1, R1, #1 @ Shift Multiplicand left (double it)
  LSR  R2, R2, #1 @ Shift Multiplier right (next bit)
  SUBS R3, R3, #1 @ Decrement Counter
  BNE  mult_loop  @ If Counter !=0, loop again
@ R0 now contains R1 * R2
```

In shift-and-add algorithms:
Step 3: Look at the Least Significant Bit (LSB) of the Multiplier. (See steps in the slides).

Q1: What is the Multiplier in 32-bit data for the machine? (Please fill in each block of 4 bits)
Multiplier: [0000][0000] [0000][0000] [0000][0000] [0001][0100]
Q2: What is the LSB of the Multiplier? [0]
Q3: What to do next? [LSL R1, R1]